LABORATOIRE DEVOPS / 2026-2027

Compréhension de l’écosystème Docker

Conteneurisation et orchestration de microservices

Application **Visit-Counter** : Flask, Redis, Nginx et Jenkins.

Réalisé par : Rayenne Abid
Classe : RT5 - INSAT
Encadrante : Saloua BEN YAHIA
Date de préparation : 14 octobre 2026

État du livrable

Le code, les tests, la configuration Docker et le pipeline sont préparés. Les tests unitaires ont été exécutés. Docker et Jenkins étant absents de l’environnement de préparation, les preuves de conteneurisation et de publication restent à produire sur la machine de l’étudiant.

Livrable	État
Application et 14 tests unitaires	Vérifiés en Python 3.12
Dockerfile, Compose, pipeline Jenkins	Préparés ; exécution Docker/CI à réaliser
Captures et lien public Docker Hub	À compléter après une exécution réelle
Partie A notée	Support anglais fourni ; tutoriel enseignant à réaliser

Le PDF joint à l’énoncé est un exemple déjà rédigé par un autre groupe. Ses auteurs, captures et URL Docker Hub ne sont pas utilisés comme preuves de ce travail.

01 / Architecture et application

Le navigateur contacte un point d’entrée Nginx. Le proxy répartit les visites sur trois instances du service web, qui utilisent toutes le même Redis. La clé **hits** est incrémentée par INCR, opération atomique côté Redis.

Composant	Rôle
Nginx / lb	Un port hôte ; répartition entre les adresses DNS web
Flask + Gunicorn / web x 3	INCR hits ; page en français et identifiant du conteneur
Redis / db-service	Stockage du compteur ; AOF dans /data
app-network	Bridge nommé ; résolution DNS entre services
redis-data	Volume nommé indépendant du cycle de vie des conteneurs

Routes et comportement

GET / ajoute une visite et affiche : « Bonjour ! Cette page a été vue X fois. Je suis le conteneur [ID] ». Le hostname fournit l’identifiant ; l’en-tête X-Container-ID permet aussi de tester la répartition. Cache-Control: no-store évite de réutiliser une réponse mise en cache.

GET /health effectue un PING Redis sans incrémenter le compteur. Une erreur Redis produit un HTTP 503, sans exposer les détails internes. Les délais de connexion et de lecture Redis sont limités à deux secondes.

Partie A / English Docker tutorial

The supplied English study guide covers images and containers, lifecycle, Dockerfiles, networking, persistence, Compose, registries and CI/CD. It includes ten self-check questions with answers.

The teacher’s Drive tutorial was not accessible during preparation. The guide is original supporting material and does not certify completion of the graded tutorial or answer questions whose content was unavailable.

Choix de conception

Gunicorn sert l’application ; le serveur de développement Flask reste réservé à un lancement local. Redis et les replicas web ne publient aucun port hôte dans Compose. Seul le proxy est accessible depuis le navigateur. Ce TP conserve un Redis et un proxy uniques : il ne fournit pas une haute disponibilité complète.

02 / Dockerfile commenté

```
# Alpine : base compacte ; nos dependances ne requierent pas de compilateur.
# Tag de version mineure : reconstruire permet de recuperer ses correctifs.
FROM python:3.11-alpine
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 APP_PORT=5000
ENV GUNICORN_CMD_ARGS="--workers=2 --threads=2 --access-logfile=- --error-logfile=-"
WORKDIR /app
# Le code change plus souvent que les dependances : conserver leur cache.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt \
    && addgroup -S appgroup && adduser -S -G appgroup -u 1000 appuser
# Le code reste possede par root et lisible, sans etre modifiable par appuser.
COPY app.py healthcheck.py .
# L'application ne tourne pas avec les droits root dans le conteneur.
USER appuser
# EXPOSE documente le port ; la publication est faite par -p ou Compose.
EXPOSE 5000
# Controle HTTP sans incrementer hits ; Redis doit aussi etre disponible.
HEALTHCHECK --interval=10s --timeout=3s --start-period=10s --retries=3 \
 CMD python healthcheck.py
# sh est remplace par Gunicorn via exec pour recevoir SIGTERM.
CMD ["sh", "-c", "exec gunicorn --bind 0.0.0.0:${APP_PORT} app:app"]
```

Justification

Python 3.11 Alpine est une base compacte adaptée aux dépendances retenues. Aucun compilateur n’est installé. La taille doit être mesurée par docker images après le build ; aucune taille supposée n’est présentée comme résultat.

requirements.txt est copié avant le code pour conserver le cache des paquets. pip --no-cache-dir évite son cache de téléchargement. USER appuser réduit les privilèges ; le code reste possédé par root. Le mode lecture seule, tmpfs /tmp et cap_drop sont appliqués par Compose.

```
docker build -t counter-app:1.0 .
docker images counter-app:1.0
docker run --rm --entrypoint id counter-app:1.0
```

03 / Réseau, volume et robustesse

Le déploiement manuel permet d’observer séparément le réseau bridge, la résolution de noms et le stockage persistant. Les commandes suivantes s’exécutent avant le déploiement Compose, pour éviter les conflits de noms.

```
docker network create app-network
docker volume create redis-data
docker run -d --name db-service --network app-network \
  -v redis-data:/data redis:8-alpine \
  redis-server --appendonly yes --appendfsync always
docker exec db-service redis-cli ping
docker run -d --name counter-web --network app-network \
  -p 80:5000 counter-app:1.0
docker network inspect app-network
```

Pourquoi AOF ?

Redis garde le compteur en mémoire. Le volume seul ne sauvegarde pas chaque incrément. AOF journalise les écritures dans /data. appendfsync always demande une synchronisation à chaque écriture avant confirmation, au prix d’un débit inférieur. Ce choix favorise le test de suppression forcée exigé par le TP.

Protocole du test de persistance

Fermer le navigateur pendant la comparaison. Lire hits, supprimer uniquement db-service, puis recréer Redis avec le même volume et les mêmes options. Attendre PONG ; hits doit être identique. Une visite suivante doit ajouter 1.

```
docker exec db-service redis-cli GET hits
docker rm -f db-service
docker run -d --name db-service --network app-network \
  -v redis-data:/data redis:8-alpine \
  redis-server --appendonly yes --appendfsync always
docker exec db-service redis-cli ping
docker exec db-service redis-cli GET hits
```

Un volume persistant ne constitue pas une sauvegarde. Ne pas supprimer redis-data pendant ce test. Le script scripts/verify_docker.py reproduit la comparaison pour le déploiement Compose et enregistre les valeurs observées.

Transition vers Compose

```
docker rm -f counter-web db-service
docker network rm app-network
```

Le volume est conservé. Compose pourra le réutiliser ; son nom explicite reste redis-data. Les réseaux et volumes aux noms fixes limitent ce TP à une seule installation simultanée sur un même moteur Docker.

04 / Docker Compose final

```
services:
  db:
    image: redis:8-alpine
    container_name: db-service
    command: ["redis-server", "--port", "${REDIS_PORT:?}", "--appendonly", "yes", "--appendfsync",
"always"]
    environment:
      REDIS_PORT: ${REDIS_PORT:?}
    volumes:
      - redis-data:/data
    networks:
      app-network:
        aliases: [db-service]
    healthcheck:
      test: ["CMD-SHELL", "redis-cli -p $$REDIS_PORT ping | grep -q PONG"]
      interval: 5s
      timeout: 3s
      retries: 10
    restart: unless-stopped

  web:
    build: .
    image: counter-app:1.0
    environment:
      REDIS_HOST: db-service
      REDIS_PORT: ${REDIS_PORT:?}
      APP_PORT: ${APP_PORT:?}
    depends_on:
      db:
        condition: service_healthy
    expose:
      - "${APP_PORT:?}"
    networks: [app-network]
    read_only: true
    tmpfs: [/tmp]
    security_opt: ["no-new-privileges:true"]
    cap_drop: [ALL]
    restart: unless-stopped
```

Les paramètres obligatoires proviennent du fichier .env fourni. Redis est unique et healthy avant le démarrage web. Les replicas n’ont ni container_name fixe ni port hôte publié. Ils peuvent donc être multipliés.

05 / Proxy et mise à l’échelle

```
  lb:
    image: nginx:1.28-alpine
    ports:
      - "${HOST_PORT:?}:8080"
    environment:
      APP_PORT: ${APP_PORT:?}
      NGINX_ENVSUBST_FILTER: APP_PORT
    volumes:
      - ./nginx/default.conf.template:/etc/nginx/templates/default.conf.template:ro
    depends_on:
      web:
        condition: service_healthy
    networks: [app-network]
    restart: unless-stopped

networks:
  app-network:
    name: app-network
    driver: bridge
volumes:
  redis-data:
    name: redis-data

COMPOSE_PROJECT_NAME=visit-counter
HOST_PORT=80
APP_PORT=5000
REDIS_PORT=6379
DOCKERHUB_USERNAME=CHANGE_ME
IMAGE_TAG=1.0

docker compose config
docker compose up -d --build --scale web=3
docker compose ps
```

Le conflit de port et sa solution

Publier le même couple adresse/port hôte pour chaque replica entraîne un conflit de bind. Chaque conteneur peut écouter sur son port interne 5000, mais les trois ne peuvent pas réserver le port 80 de l’hôte. Le proxy réserve ce port une seule fois et répartit les requêtes.

Nginx 1.28 utilise un upstream partagé, le resolver Docker et resolve pour actualiser les adresses de web. Le filtre envsubst APP_PORT préserve les variables Nginx. Le fichier nginx/default.conf.template est livré dans le ZIP.

06 / Pipeline Jenkins et publication

Un job Pipeline from SCM suit la branche ***/main** et lit le Jenkinsfile du dépôt. Le trigger githubPush est associé au webhook GitHub : https://VOTRE_JENKINS/github-webhook/. Un premier build initialise le job.

Étape	Action et condition
Checkout	Récupérer la révision de main
Build application	Créer un venv, installer les dépendances, compiler app.py
Unit tests	Exécuter pytest ; publier le rapport JUnit
Build image	docker build --pull ; seulement sur main
Publish Docker Hub	Login via token ; push tag build-commit et latest

Configuration requise

L'agent Linux doit fournir Git, Python avec venv, Docker CLI et un accès au daemon. Plugins : Pipeline, Git, GitHub, Credentials Binding, JUnit et Timestamper. Le job est limité à main. Jenkins doit être joignable depuis GitHub pour recevoir les événements push.

Créer un credential Username with password, ID **dockerhub-credentials** : username = compte Docker Hub ; password = token autorisant le push. Créer un dépôt public counter-app dans ce compte. Le token reste dans withCredentials, sans interpolation Groovy ; DOCKER_CONFIG temporaire est supprimé même si le push échoue.

Publication manuelle équivalente

```
docker login -u VOTRE_COMPTE
docker tag counter-app:1.0 VOTRE_COMPTE/counter-app:1.0
docker push VOTRE_COMPTE/counter-app:1.0
```

Lien Docker Hub à compléter : aucune publication effectuée ici. Le modèle est https://hub.docker.com/r/VOTRE_COMPTE/counter-app.

Le Jenkinsfile intégral et les instructions du webhook figurent dans le projet. La présence d'un pipeline ne prouve pas son exécution : ajouter la capture Jenkins et vérifier le push après un commit sur main.

07 / Vérifications et preuves

Résultat réel : 14 tests exécutés ; 0 échec/erreur. Rapport : evidence/unit-tests.xml.

Les tests couvrent les incréments successifs, le compteur partagé par deux instances applicatives, le hostname, la sonde sans visite, les erreurs de connexion et de timeout, une valeur Redis invalide, le rendu échappé et la configuration par variables d'environnement. Redis est simulé par fakeredis.

La compilation Python, la syntaxe des scripts Bash et la structure YAML ont également été contrôlées. Les tests unitaires ne prouvent ni le build d'image, ni la distribution Nginx réelle, ni la persistance sur disque.

Intégration Docker : non exécutée ici, Docker étant absent. Lancer python scripts/verify_docker.py après le démarrage des trois replicas.

Captures obligatoires

docker-images.png : à ajouter - Taille de counter-app:1.0.

network-inspect.png : à ajouter - app-network et conteneurs connectés.

browser.png : à ajouter - Page réelle dans le navigateur.

Après exécution, placer les trois captures PNG dans evidence/, ajouter dockerhub-url.txt puis relancer python docs/build_report.py. Le rapport insère alors les images. Il ne substitue aucun exemple aux captures réelles.

Commande de collecte

```
python scripts/verify_docker.py
```

Le script exige trois identifiants, vérifie les UID non-root, compare le compteur après suppression forcée puis vérifie la visite suivante. Il collecte aussi docker images, network inspect et compose ps en texte.

08 / Réflexion, nettoyage et sources

RUN et CMD

RUN s’exécute pendant docker build et modifie le système de fichiers de l’image. Exemple : installer requirements.txt. **CMD** définit la commande par défaut au démarrage, ici Gunicorn. Elle peut être remplacée par une commande fournie à docker run et ne s’exécute pas au build.

Nettoyage en une commande

```
docker compose down --rmi all --volumes --remove-orphans
```

Cette commande retire les conteneurs du projet, le réseau, les volumes déclarés et les images des services si elles ne sont pas utilisées ailleurs. Le compteur est supprimé avec redis-data. Pour conserver les données, utiliser simplement docker compose down.

docker system prune -a --volumes nettoie les ressources globalement inutilisées ; il laisse les conteneurs actifs et ne supprime pas tous les volumes nommés. Ce n’est donc pas un remplacement du nettoyage Compose.

Éléments à finaliser avant remise

Réaliser le tutoriel A noté ; exécuter le build et les tests Docker ; ajouter les trois captures obligatoires ; pousser une image publique dans le compte personnel ; vérifier le webhook et le pipeline Jenkins ; régénérer ce rapport. Le dossier fourni couvre le code et les procédures des parties B et C.

Documentation officielle utilisée

Dockerfile :
https://docs.docker.com/reference/dockerfile/

Services Compose :
https://docs.docker.com/reference/compose-file/services/

Réseaux Compose :
https://docs.docker.com/compose/how-tos/networking/

Nettoyage Docker :
https://docs.docker.com/reference/cli/docker/system/prune/

Persistance Redis :
https://redis.io/docs/latest/management/persistence/

Upstream Nginx :
https://nginx.org/en/docs/http/ngx_http_upstream_module.html

Pipeline Jenkins :
https://www.jenkins.io/doc/book/pipeline/

Credentials Binding :
https://www.jenkins.io/doc/pipeline/steps/credentials-binding/